World Models for Video Prediction in Procgen Games

Dev Paresh Chaudhari
Dept. of Software Engineering
Western University
London, Ontario
dchaud26@uwo.ca

Tejanvesh Gangavarapu
Dept. of Software Engineering
Western University
London, Ontario
tgangava@uwo.ca

Obaid Mohiuddin
Dept. of Software Engineering
Western University
London, Ontario
omohiudd@uwo.ca

Kwaku Asare
Dept. of Software Engineering
Western University
London, Ontario
kasare2@uwo.ca

***Abstract*—Predicting future visual states in procedurally generated environments needs world models that can go beyond just remembering layouts. This study combines a VQ-VAE tokenizer, which converts visual frames into discrete token sequences, with an autoregressive transformer that predicts future tokens as a sequence modelling problem, following the approach introduced by IRIS, to forecast future frames in Procgen We train on levels 0 to 99 and evaluate levels 100 and higher. The VQ-VAE compresses 64x64 frames into 8x8 grids of discrete tokens taken from a 512-entry codebook. We test four transformer setups, change their capacity and regularization settings, and evaluate them at prediction horizons of 1, 5, 10, and 20 steps using PSNR, SSIM, and LPIPS. The results indicate that the best setup depends on the environment. For instance, on CoinRun, heavy regularization cuts the generalization gap from over 4 dB to below 1.34 dB and outperforms the highest-capacity model on unseen levels. In contrast, on StarPilot, all setups generalize similarly. However, to achieve both high accuracy and strong generalization, we need more diverse training data instead of relying solely on regularization. Future work will focus on using diffusion-based predictions to fix compounding errors and integrating reinforcement learning. Key lessons learned include the importance of iterative development, tracking early validation, and fixing codebook collapse through design rather than training tricks. To our knowledge, this is the first evaluation of an IRIS-style world model on procedurally generated environments with separate training and test layouts.**

Keywords—world models, VQ-VAE, autoregressive transformer, video prediction, procedural generation, generalization, discrete tokenization, Procgen, codebook collapse, regularization

I. Introduction

Predicting future environmental states from visual data is a fundamental challenge in computer vision, with applications in environmental simulation, visual reasoning, and autonomous decision-making. World models address this problem by learning internal representations that capture the dynamics of an environment, enabling prediction of future states given past observation and actions [1]. Recent approaches based on discrete tokenization and autoregressive transformers have achieved strong predictive accuracy in fixed game environments [2]-[4], but whether can such models generalize to environments whose visual layouts were never observed during training has not been established.

Procedurally generated game environments provide a strong setting for studying this question. Procgen [5] generates each episode from a unique random seed, producing levels that share visual building blocks such as platform tiles, character sprites, and background textures, but arrange them in distinct spatial configurations. A model trained on levels 0 through 99 and evaluated on levels 100 onward must rely on learned dynamics rather than memorized layout patterns, making Procgen a meaningful benchmark for visual generalization. However, most existing world models represent observations as continuous, high-dimensional latent vectors, which are difficult to model sequentially and prone to overfitting in settings where visual layouts vary systematically across episodes.

Discrete tokenization provides a principled response to these limitations. Vector Quantized Variational Autoencoders (VQ-VAEs) convert raw visual observations into grids of discrete token indices drawn from a learned visual codebook [3], providing an effective interface between visual perception and sequence modelling. Micheli et al [2] demonstrated that a transformer trained autoregressively over such token sequences can serve as a world model, achieving human-level performance in Atari with significantly fewer environment interactions. Subsequent work has extended this paradigm to interactive environment generation [4] and real-time neural game simulation [6], further validating discrete tokenization as a general-purpose representation for learned dynamics models.

Despite these advances, these models have been evaluated exclusively on Atari, where level layouts are fixed and identical across training and evaluation. Whether the discrete tokenize-then-predict paradigm generalizes procedurally generated environments with variable layouts has not been tested. Furthermore, the relationship between model capacity, regularization, and generalization in autoregressive video prediction remains uncharacterized.

To address this gap, we train a VQ-VAE and a transformer-based video prediction model for Procgen and conduct a systematic ablation across four transformer configurations. Our objective is to, first, train the VQ-VAE tokenizer on gameplay observations from CoinRun and StarPilot to assess the viability of discrete tokenization as a visual representation under procedural variability; second, design an autoregressive transformer that predicts future frame tokens conditioned on player actions; third, ablate model capacity (embedding dimension 256 to 384, layers 4 to 8) and regularization (dropout 0.1 to 0.3, weight decay, label smoothing) across four variants; and lastly, evaluate prediction quality on unseen levels at horizons 1, 5, 10, and 20 steps, using Learned Perceptual Image Patch Similarity (LPIPS) [8], Peak Signal-to-Noise Ratio

(PSNR), and Structural Similarity Index (SSIM) [8] as evaluation metrics.

II. Background & Related Work

World Models, first introduced conceptually in the 1943 paper by Kenneth Craik [9], proposed the hypothesis that the mind constructs internal models to understand and predict how the world around it works. This principle has influenced AI research, moving from early symbolic reasoning systems such as SHRDLU [10] that maintained explicit world representations, through Schmidhuber's [1] proposal for learning environment dynamics with recurrent neural networks. The modern formulation by Ha and Schmidhuber [1], decomposes the system into three components, a vision model, a memory model and a controller. The vision model is responsible for compressing visual observations into latent representation, which is used by the recurrent neural network-based memory model to learn temporal dynamics within the latent space. The controller role is to execute actions based on learned representation provided by the memory model. Their findings showcased that an agent could learn useful behaviour entirely within their own simulated environments.

A key challenge in developing such systems is representing visual observations for sequential prediction. Standard autoencoders produce continuous latent vectors, which work well with recurrent dynamic models but cause issues for autoregressive architectures that rely on discrete vocabularies. Van den Oord et al. [3] tackled this with VQ-VAE, which uses a learnable codebook of embedding vectors. Each spatial position in the encoder output is replaced by the index of the nearest codebook entry, calculated using Euclidean distance. This converts a continuous feature map into a grid of discrete integer tokens. The decoder then reconstructs images from these quantized representations. Training combines reconstruction loss with a commitment loss to stabilise encoder-codebook alignment, while updating codebook vectors through exponential moving averages of the assigned encoder outputs instead of gradient descent. Gradients pass through the non-differentiable quantization step using a straight-through estimator. This discretisation allows transformer-based sequence models to work with visual data. The prediction target becomes a categorical distribution over a limited set of codebook entries, rather than a continuous pixel array. A persistent challenge is codebook collapse, where most entries receive no assignments and the model uses only a portion of its representational capacity. Effective solutions include reducing the embedding dimension to make the codebook space compact enough for EMA to handle, resetting unused entries to random encoder outputs, and using fixed grid quantization instead of learned codebooks.

Micheli et al. [2] combined discrete tokenization with autoregressive sequence modelling to create IRIS, a world model that frames visual dynamics prediction as a language modelling problem. Each 64×64 game frame is converted into an 8×8 grid of discrete tokens, resulting in 64 tokens per frame from a codebook of 512 entries. Frames and actions are combined into a flat sequence, and a GPT-style causal transformer predicts the next token at each position, based on all preceding tokens through masked self-attention. On the Atari 100k benchmark, IRIS achieved human-level performance on 10 out of 26 games using about two hours of gameplay data. This demonstrated that the discrete tokenize then predict method is highly efficient with data. Delta-IRIS later reduced computational costs by tokenizing only the differences between frames instead of the full frames, cutting down the sequence length the transformer must process at each step.

DIAMOND [11] takes a different direction to tackle the same problem. Instead of compressing observations into discrete tokens, it predicts future frames directly in continuous pixel space using denoising diffusion. Given previous frames and an action, the model gradually refines a noise sample into the predicted next frame. Since no information is lost during quantization, DIAMOND retains visual details that VQ-VAE tokenization might miss. The authors showed that these details are significant as small prediction errors can accumulate during long autoregressive rollouts, gradually reducing prediction quality. Both IRIS and DIAMOND, created by the same research team, represent the two main paradigms for learned visual simulation, discrete autoregressive generation and continuous diffusion-based generation.

The transformer [12] is the shared backbone underlying the IRIS framework. Multi-head self-attention allows each token in a sequence to focus on all preceding tokens, capturing long range dependencies that recurrent architectures handle less effectively. For autoregressive generation, a causal mask sets attention weights to zero for all future positions, ensuring that predictions depend only on past context. The capacity of a transformer is governed by its embedding dimension, number of layers, and feed-forward width. Regularisation techniques, such as dropouts, weight decay, label smoothing, control how that capacity is utilised. Dropout forces redundant representations by randomly zeroing activations during training. Weight decay adds an L2 penalty that biases the model toward smaller parameter values. Label smoothing redistributes probability from the target token to all tokens, discouraging overconfident predictions. The interaction between these techniques and model capacity has been studied extensively for classification tasks, but their effect on autoregressive visual generation, where prediction coherence across a 64-token frame matters as much as individual token accuracy, is still unknown.

Procgen [5] is a benchmark suite of procedurally generated games built on top of OpenAI's Gymnasium framework. Unlike Atari, where each game has a fixed layout that remains the same across training and evaluation, Procgen generates a new level from a random seed at every episode. Levels reuse shared visual elements such as platform tiles, character sprites, and background textures, but arrange them into different spatial configurations each time. This means that training on levels 0 through 99 and evaluating on levels 100 onward creates a shift in layout, while keeping the visual vocabulary the same. A model that performs well on unseen levels has learned transferable dynamics, while one that only performs well on training levels has memorised specific spatial patterns.

We use two Procgen games for this work, namely, CoinRun and StarPilot. CoinRun is a side-scrolling platformer with significantly different platform positions, gap locations, and obstacle placements across levels. StarPilot is a vertically

scrolling shooter where backgrounds move uniformly and entities follow similar patterns regardless of the level seed. This combination allows us to test whether the capacity, regularisation balance varies based on the structural diversity of the environment, specifically whether games with more level specific variation.

To our knowledge, no previous research evaluates IRIS-style video prediction in procedurally generated environments where training and test layouts differ. The link between model capacity, regularization strength, and visual generalization in autoregressive video prediction is also not explored. We address these gaps through a controlled study across four transformer configurations on two structurally different Procgen games.

III. Methods

A. Research Plan

To see if an IRIS-style world model can work with new visual layouts, we need a structured process that separates visual representation from dynamics prediction. The system includes three parts: a VQ-VAE tokenizer that changes raw 64×64 game frames into 8×8 grids of discrete tokens, an autoregressive transformer that predicts future token sequences based on player actions, and the VQ-VAE decoder that rebuilds frames from predicted tokens. We train the tokenizer once and keep it the same for all experiments. This way, any differences in prediction quality come from the transformer, not the visual representation.

The main focus of this work is a comparative study of four transformer configurations. Each version varies in embedding dimension, depth, dropout rate, context window length, and whether label smoothing is used. All four are trained on tokenised gameplay from levels 0 to 99 and evaluated on levels 100 and up, for both CoinRun and StarPilot. This setup allows us to directly measure how capacity and regularization impact generalization since the only thing that changes between experiments is the transformer itself.

We measure performance using PSNR, SSIM [7], and LPIPS [8] for pixel-level reconstruction quality, and token accuracy for discrete prediction correctness. These metrics are calculated at prediction horizons of 1, 5, 10, and 20 steps. Comparing results between seen and unseen levels highlights the generalisation gap for each configuration. Additionally, comparing across the two games indicates whether the best configuration relies on the amount of structural variation in the environment

B. Research Objectives

The proposed system is evaluated across four staged objectives, each targeting a distinct component of the pipeline.

Research Question: Can an IRIS-style discrete tokenize-then-predict world model, trained on a fixed set of procedurally generated levels, produce accurate multi-step frame predictions on unseen level layouts, and how does the balance between model capacity and regularisation affect this generalization?

Hypothesis: Of the four transformer configurations, the variants with moderate capacity and regularisation will generalise most effectively to unseen levels, while the highest-capacity variant without regularisation will perform best on training levels but show the largest generalisation gap.

O1: Train a VQ-VAE tokenizer for Procgen game frames. Train a VQ-VAE to compress 64×64 gameplay frames into 8×8 grids of discrete tokens from a codebook of 512 learned visual embeddings. This establishes the shared visual representation used across all experiments and determines whether discrete tokenization preserves enough visual detail under the diversity of procedurally generated environments.

O2: Autoregressive Prediction of Future Visual States. Train and compare four transformer variants of differing depth, embedding dimension, dropout, and context window length to predict future discrete token sequences conditioned on action inputs. Evaluate predictive performance across horizons of 1, 5, 10, and 20 steps to determine which architectural configuration best captures the temporal dynamics of each environment.

O3: Ablate the capacity–regularisation trade-off. Train four transformer configurations spanning embedding dimensions from 256 to 384, depths from 4 to 8 layers, dropout from 0.1 to 0.3, and with or without label smoothing and weight decay. This directly tests whether high-capacity models that memorise training levels can still generalise to new ones, or whether regularisation is needed at the cost of lower training performance.

O4: Evaluate prediction quality on unseen levels at multiple horizons. Test all four configurations on levels withheld from training at horizons of 1, 5, 10, and 20 steps, using PSNR, SSIM, LPIPS, and token accuracy. This measures the generalisation gap for each configuration and reveals whether the optimal balance between capacity and regularisation depends on the environment.

These objectives form a staged evaluation pipeline: the first objective assesses the quality of the discrete visual representation, the second objective assesses the capacity of the sequence model to predict future states, the third objective investigates the architectural factors that influence prediction quality, and the final objective determines whether both components generalise beyond the training distribution.

C. Research Methodology

Data Collection: Gameplay data was collected from two Procgen environments, CoinRun and StarPilot, using the procgen library with Gymnasium [14]. A random policy was used to collect transitions rather than a trained agent, ensuring diverse and unbiased state coverage across the observation space. A trained policy would bias the data toward high reward trajectories, leaving large portions of the state space unrepresented. For each game, 100,000 transition frames were collected from training levels (levels 0–99) and 20,000 transition frames from test levels (levels 100–199). Each transition consists of a 64×64 RGB frame, the action taken, the resulting next frame, the reward, and an episode termination flag. Both games use a discrete action space of 15 actions. All data was stored in HDF5 format with gzip compression. The training and test level ranges are non-overlapping, ensuring the model never observes the spatial layouts present in the test set during training.

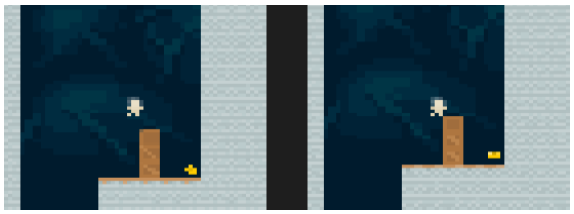

Fig. 1: Raw Image Data - CoinRun

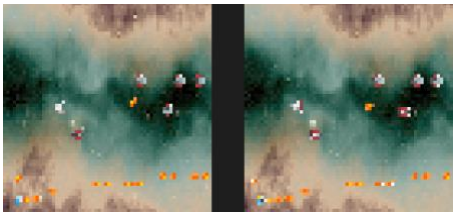

Fig. 2: Raw Image Data - StarPilot

VQ-VAE Tokenizer: The encoder has three strided convolutional layers (stride 2) with residual blocks and batch normalization [15]. It compresses a 64×64×3 input frame into an 8×8×64 feature map. The vector quantizer uses a codebook with 512 entries, each with an embedding dimension of 64. This codebook updates through exponential moving averages (EMA) with a decay of 0.99 instead of gradient descent [3]. The decoder mirrors the encoder with transposed convolutions, reconstructing the 8×8×64 quantized representation back to 64×64×3. This results in a compression ratio of 12,288-pixel values to 64 discrete token IDs, which is a reduction of 192 times.

Training: Training uses Adam [16] with a learning rate of 3e-4, a batch size of 64, and runs for 30 epochs on the combined CoinRun and StarPilot training data. The loss function combines MSE reconstruction loss with a commitment loss weighted by 0.25, which stabilises the alignment between encoder outputs and codebook vectors. The codebook itself is updated via EMA rather than through the loss function. A straight-through estimator passes gradients through the non-differentiable quantization step [17].

An initial configuration using an embedding dimension of 512 with 1,024 codebook entries suffered from codebook collapse, with only 1–5% of entries receiving assignments. The larger embedding space made it difficult for EMA updates to maintain coverage across all codebook entries, causing the model to concentrate on a small number of vectors. Reducing the embedding dimension to 64 and the codebook size to 512 resolved this, as the smaller embedding space is compact enough for EMA to cover effectively. This produced stable training with over 90% codebook utilisation and a reconstruction loss of approximately 0.004. Codebook resets were also attempted but caused instability, as the decoder could not handle sudden changes to codebook entries it had already adapted to. This approach was abandoned in favour of the smaller embedding configuration.

After training, the VQ-VAE is frozen and used to tokenize all gameplay frames into sequences of 64 discrete token IDs per frame. These tokenized sequences, along with their corresponding actions and episode termination flags, are stored in separate HDF5 files and used as inputs for the transformer.

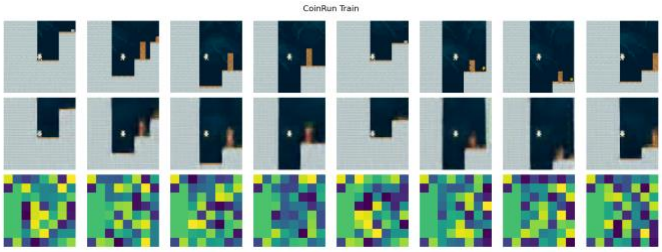

Fig. 3: CoinRun VQ-VAE output showing original frames (top), decoder reconstructions (middle), and 8×8 discrete token maps (bottom).

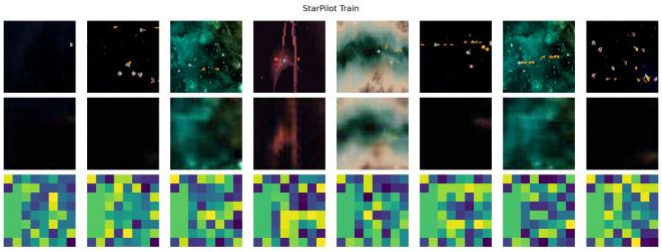

Fig. 4: StarPilot VQ-VAE output showing original frames (top), decoder reconstructions (middle), and 8×8 discrete token maps (bottom).

Transformer Dynamics Model: The transformer takes as input an interleaved sequence of frame tokens and action tokens from the tokenizer and predicts the next token at each position using causal self-attention. Each frame contributes 64 tokens (the flattened 8×8 grid), and each action is represented as a single token offset by the visual vocabulary size (512) to avoid collisions with visual token IDs. For a context window of 4 frames and 1 target frame, the full sequence is: [Frame_1 tokens | Action_1 | Frame_2 tokens | Action_2 | ... | Frame_4 tokens | Action_4 | Frame_5 tokens], giving a total sequence length of 324 tokens ($5 \times 64 + 4 \times 1$). A context window of 4 frames was chosen to provide sufficient temporal context for predicting short-term dynamics while keeping the sequence length manageable for attention computation.

Positional information is encoded through learned temporal and spatial embeddings. The temporal embedding identifies which timestep each token belongs to, while the spatial embedding identifies the token's position within the frame (0–63) or marks it as an action token (position 64). These two embeddings are summed and added to the token embeddings before being passed through the transformer layers.

Each transformer layer consists of multi-head self-attention with a causal mask, followed by a feed-forward network with GELU activation [18]. Layer normalisation and residual connections are applied after both the attention and feed-forward stages. The output projection maps the final hidden states to a distribution over the 512 visual token vocabulary using a linear layer. The loss is computed as cross-entropy over frame token positions only, excluding action token positions from the loss computation. Gradient clipping with a maximum norm of 1.0 is applied to prevent training instability.

Training sequences are extracted from the tokenized data by sliding a window of context_frames + 1 consecutive frames within each episode. Sequences that span episode boundaries

(where a termination flag occurs within the window) are excluded. Training and validation data are loaded from seen levels (0–99) and unseen levels (100–199) respectively, with a batch size of 32 for all configurations.

During inference, the model generates the next frame one step at a time. It considers past frames and actions to predict each of the 64 tokens in the next frame, feeding each predicted token back into the sequence before predicting the next one. V1 uses argmax decoding, which means it always picks the highest-probability token for each position. V2 through V4 use categorical sampling with temperature scaling. In this method, the logits are divided by a temperature parameter before sampling from the resulting distribution. While argmax decoding is predictable, it can lead to mode collapse. This happens when the model keeps choosing the same safe tokens, resulting in blurred or repetitive predictions. Categorical sampling keeps the predicted frames diverse, but it can lead to some incorrect tokens. For multi-step prediction, the fully generated frame is added to the context window, and the process repeats, updating the context to maintain the fixed window size.

Capacity–Regularisation Ablation: The four transformer configurations were developed iteratively, with each version informed by the results of the previous one.

V1 was designed as a high-capacity baseline with minimal regularisation to establish an upper bound on training performance. It uses an embedding dimension of 384 with 8 layers, 8 attention heads, a feed-forward dimension of 1,536, and a dropout rate of 0.1. It was trained with Adam at a learning rate of 3e-4 in two phases: 30 epochs at a constant learning rate, followed by 30 additional epochs with cosine annealing. No weight decay, label smoothing, or validation monitoring was used. Training was performed on an A100 GPU. V1 reached 91% training accuracy, but with no validation set there was no way to measure the generalization gap during training.

V2 was built in direct response to V1's suspected overfitting. Capacity was reduced to an embedding dimension of 256 with 4 layers, and heavy regularisation was applied: dropout was increased to 0.3, weight decay of 0.01 was added through AdamW [19], and label smoothing of 0.1 was introduced to discourage overconfident token predictions. A ReduceLROnPlateau scheduler was used to halve the learning rate after 3 epochs without validation improvement, and early stopping with a patience of 8 epochs was applied. This was the first configuration to use a validation set from unseen levels. Training was performed on an A100 GPU. V2 achieved 72% validation accuracy with almost no train-val gap, but the low training accuracy suggested underfitting.

V3 aimed to find a middle ground between V1 and V2. It uses an embedding dimension of 256 with 6 layers and a dropout rate of 0.2, keeping weight decay and label smoothing from V2 but using a cosine annealing scheduler instead of ReduceLROnPlateau, and increasing the learning rate to 3e-4. Early stopping patience was extended to 10 epochs to allow more training time. Training was performed on a G4 GPU and took approximately 4 hours. V3 achieved 75% validation accuracy with a validation loss of 1.64.

V4 tests whether weight decay alone is sufficient to regularise a high-capacity model, without relying on heavy dropout or label smoothing. It matches V1's capacity (d_model=384, 8 layers) but adds weight decay of 0.01 through AdamW and extends the context window from 4 to 5 frames. Dropout remains at 0.1 and no label smoothing is applied. Cosine annealing is used over 30 epochs. Training was performed on a G4 GPU. V4 achieved 72% validation accuracy with a validation loss of 0.84.

Each model took between 2 and 4 hours to train depending on the configuration and GPU used. All four configurations share the same frozen VQ-VAE tokenizer, the same data splits, and the same evaluation protocol.

Parameter	V1	V2	V3	V4
d_model	384	256	256	384
Layers	8	4	6	8
d_ff	1536	1536	1536	1536
Heads	8	8	8	8
Dropout	0.1	0.3	0.2	0.1
Context Frames	4	4	4	5
Optimizer	Adam	Adam W	Adam W	Adam W
Learning Rate	3e-4	1e-4	3e-4	3e-4
Weight Decay	–	0.01	0.01	0.01
Label Smoothing	–	0.1	0.1	–
Scheduler	Cosine	Reduce LROnP lateau	Cosine	Cosine
Epochs	30 + 30	40	60	30

Table 1: Training parameters for the four transformer configurations

Evaluation: All four models are evaluated on both seen (training) and unseen (test) levels for CoinRun and StarPilot. For each condition, 200 sequences are sampled uniformly from valid starting positions, where a valid position is one that allows at least 20 consecutive frames within a single episode without a termination flag.

At each prediction horizon (1, 5, 10, and 20 steps), the model predicts future frames autoregressively starting from a context window of ground-truth frames and actions. Predicted token grids are decoded back to pixel space using the frozen VQ-VAE decoder, and metrics are computed on the final predicted frame at each horizon.

Four metrics are used for evaluation. PSNR measures pixel-level reconstruction accuracy in decibels, where higher is better. SSIM measures structural similarity between predicted and ground-truth frames, where higher is better. LPIPS [8] uses a pretrained AlexNet to measure perceptual distance, where lower is better. Token accuracy measures the fraction of predicted tokens that exactly match the ground-truth tokens, providing a direct measure of the transformer's discrete prediction performance independent of the decoder.

Comparing metrics between seen and unseen levels at each horizon shows the generalisation gap for each configuration. Comparing across CoinRun and StarPilot reveals whether the best configuration depends on the environment.

IV. RESULTS

A. One-Step Prediction

At a prediction horizon of 1 step, V1 produces the best results on CoinRun training levels. It achieves a PSNR of 24.02 dB, an SSIM of 0.967, an LPIPS of 0.081, and a token accuracy of 86.9%. This is the highest single-step performance of any setup under any condition. It reflects V1's strong capacity and its ability to remember training-level patterns. However, on unseen CoinRun levels, V1 drops to a PSNR of 19.83 dB and a token accuracy of 51.4%. This leads to a generalisation gap of 4.19 dB. V2 achieves 21.99 dB on seen levels and 20.65 dB on unseen levels, reducing the gap to 1.34 dB. V3 and V4 have similar gaps of 1.29 dB and 1.87 dB, respectively.

On StarPilot, all four models produce similar one-step PSNR values ranging from 21.43 to 21.77 dB across both seen and unseen levels. The generalization gaps are close to zero or slightly negative for all setups. This indicates that StarPilot's consistent scrolling structure does not create a significant shift between training and test levels at short horizons. However, SSIM values on StarPilot are noticeably lower than on CoinRun across all models, averaging about 0.59 to 0.63 compared to 0.87 to 0.97. Additionally, LPIPS values are much higher for StarPilot, around 0.40 compared to 0.08 to 0.13. This suggests that StarPilot's small details have been omitted in the tokenization process. We discussed this problem earlier and can see the loss of small details in the reconstructed image in Fig 4.

B. Multi-Step Prediction

As the prediction horizon increases, all models perform worse, but the rate of decline varies by configuration and game. Table 2 and Table 3 show the complete results for CoinRun and StarPilot, respectively.

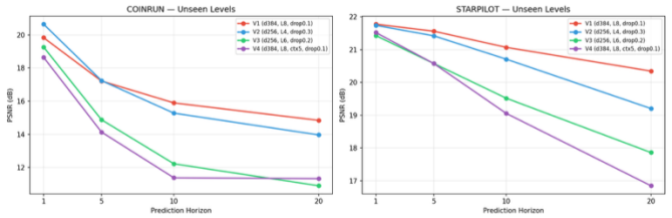

Fig. 5: Prediction Quality vs Horizon (Test Sets)

In CoinRun, V1 falls from 24.02 dB at horizon 1 to 16.40 dB at horizon 20 on seen levels, a drop of 7.62 dB. On unseen levels, it reduces from 19.83 to 14.84 dB. V2 experiences more consistent degradation, dropping from 21.99 dB to 14.08 dB on seen levels and from 20.65 to 13.97 dB on unseen levels.

The difference narrows to 0.11 dB by horizon 20. This indicates that V2's regularization [14] prevents memorisation but also restricts its ability to make accurate predictions on familiar layouts.

V3 and V4 degrade the fastest at longer horizons, going below 12 dB on CoinRun by horizon 10. This is worse than V1 and V2 in absolute terms. The likely reason is that V3 and V4 are in an unstable area of the capacity regularization space. They do not have enough capacity to memorize patterns like V1, yet they also lack the regularization strength of V2 to learn strong, transferable features. As a result, errors build more quickly during autoregressive rollouts since each mis-predicted token pushes future predictions further from the correct answer.

On StarPilot, multi-step degradation is more gradual for V1 and V2. V1 drops from 21.51 dB at horizon 1 to 20.27 dB at horizon 20 on seen levels, a loss of only 1.24 dB, and from 21.77 to 20.34 dB on unseen levels. V2 follows a similar pattern, dropping from 21.54 to 18.78 dB on seen levels. The slower degradation reflects StarPilot's uniform scrolling structure, where consecutive frames share more visual content than CoinRun's varied platform layouts. V3 and V4 degrade more steeply on StarPilot, dropping below 18 dB by horizon 20 on both seen and unseen levels.

V1				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	24.02	0.967	0.081
Seen	5	21.76	0.926	0.099
Seen	10	18.74	0.847	0.135
Seen	20	16.4	0.761	0.177
Unseen	1	19.83	0.88	0.126
Unseen	5	17.22	0.806	0.163
Unseen	10	15.9	0.759	0.178
Unseen	20	14.84	0.715	0.2
V2				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	21.99	0.929	0.099
Seen	5	17.76	0.82	0.152
Seen	10	15.18	0.737	0.194
Seen	20	14.08	0.698	0.212
Unseen	1	20.65	0.906	0.116
Unseen	5	17.24	0.817	0.163
Unseen	10	15.28	0.752	0.187
Unseen	20	13.97	0.702	0.213
V3				
Split	**H**	PSNR	SSIM	LPIPS
Seen	1	20.56	0.89	0.12
Seen	5	14.7	0.699	0.214
Seen	10	12.81	0.609	0.259
Seen	20	11.12	0.52	0.305
Unseen	1	19.27	0.868	0.134
Unseen	5	14.88	0.72	0.215
Unseen	10	12.22	0.592	0.273
Unseen	20	10.9	0.513	0.311
V4				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	20.51	0.919	0.108
Seen	5	14.51	0.731	0.206
Seen	10	11.29	0.569	0.287
Seen	20	11.33	0.555	0.29
Unseen	1	18.65	0.881	0.131
Unseen	5	14.14	0.72	0.214
Unseen	10	11.38	0.585	0.287
Unseen	20	11.33	0.561	0.303

Table 2: CoinRun prediction quality across all models, splits, and horizons

V1				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	21.51	0.632	0.411
Seen	5	21.25	0.617	0.424
Seen	10	20.87	0.604	0.427
Seen	20	20.27	0.584	0.448
Unseen	1	21.77	0.613	0.398
Unseen	5	21.56	0.603	0.405
Unseen	10	21.07	0.587	0.418
Unseen	20	20.34	0.555	0.436
V2				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	21.54	0.632	0.41
Seen	5	21.2	0.613	0.422
Seen	10	20.38	0.57	0.432
Seen	20	18.78	0.462	0.462
Unseen	1	21.74	0.611	0.4
Unseen	5	21.42	0.598	0.407
Unseen	10	20.71	0.559	0.421
Unseen	20	19.2	0.466	0.446
V3				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	21.43	0.628	0.413
Seen	5	20.3	0.564	0.436
Seen	10	19.08	0.482	0.464
Seen	20	17.46	0.375	0.51
Unseen	1	21.43	0.594	0.404
Unseen	5	20.58	0.545	0.419
Unseen	10	19.52	0.465	0.454
Unseen	20	17.86	0.352	0.502
V4				
Split	**H**	**PSNR**	**SSIM**	**LPIPS**
Seen	1	21.45	0.623	0.418
Seen	5	20.49	0.568	0.43
Seen	10	18.99	0.475	0.452
Seen	20	16.74	0.3	0.508
Unseen	1	21.52	0.594	0.408
Unseen	5	20.57	0.539	0.427
Unseen	10	19.05	0.443	0.462
Unseen	20	16.85	0.288	0.514

Table 3: StarPilot prediction quality across all models, splits, and horizons

C. Generalisation Gap Analysis:

The generalisation gap is largest for V1 on CoinRun, hitting 4.54 dB at horizon 5. V2 reduces this gap to below 1.34 dB at all horizons, and it reaches nearly zero at horizons 10 and 20. V3 and V4 show gaps of 1.29 dB and 1.87 dB at horizon 1, placing them between V1 and V2, but their rapid degradation at longer horizons means this advantage does not hold. Notably, V2 performs better than V1 on new CoinRun levels at horizon 1, with scores of 20.65 dB compared to 19.83 dB. This means the most regularized model makes better predictions on new levels than the highest-capacity model, even though it has much lower training accuracy.

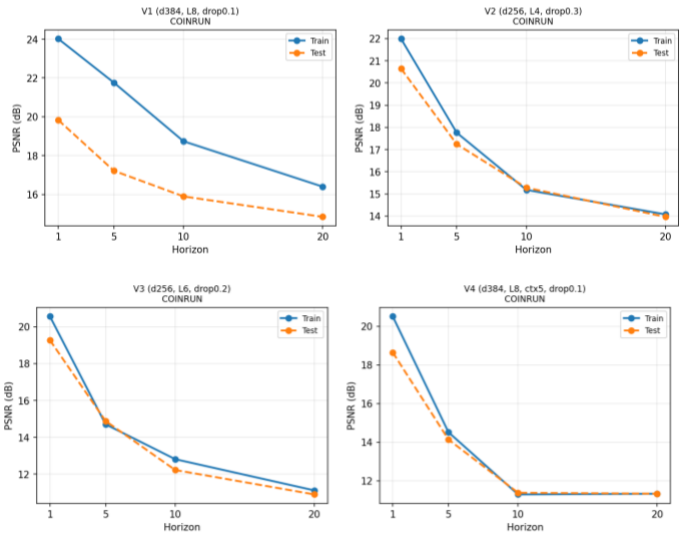

Fig. 6: CoinRun Generalization Gap – Train vs Test PSNR

On StarPilot, the generalisation gap is small for all models. Some configurations have slightly negative gaps, which means test performance is just a bit better than training performance. This shows that StarPilot does not create a significant distribution shift between training and test levels.

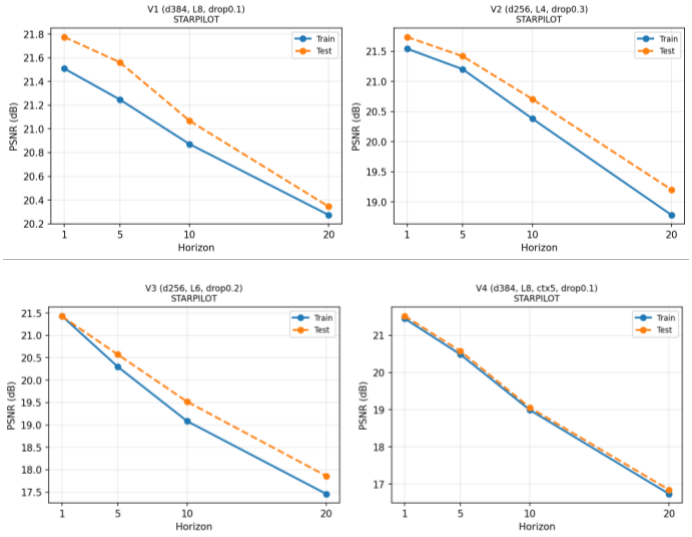

Fig. 7: StarPilot Generalization Gap – Train vs Test PSNR

While regularization clearly helps narrow the generalization gap, the regularized models (V2, V3, V4) have lower overall accuracy than V1 on training data. To further close this gap and achieve both high accuracy and strong generalization, training on more levels would likely be necessary. This way, the model can experience enough layout diversity to learn general dynamics without needing heavy regularization to avoid memorization.

Figures 8 and 9 show predicted frames from the CoinRun and StarPilot test sets at steps 9 and 17 of the model's dreams. On CoinRun, V1 and V2 preserve the scene layout while V4 produces visible artifacts, showcasing how compounding errors affect intermediate configurations. On StarPilot, all models

capture the broad spatial structure but lose fine detail, consistent with the information loss introduced by VQ-VAE tokenization at this resolution.

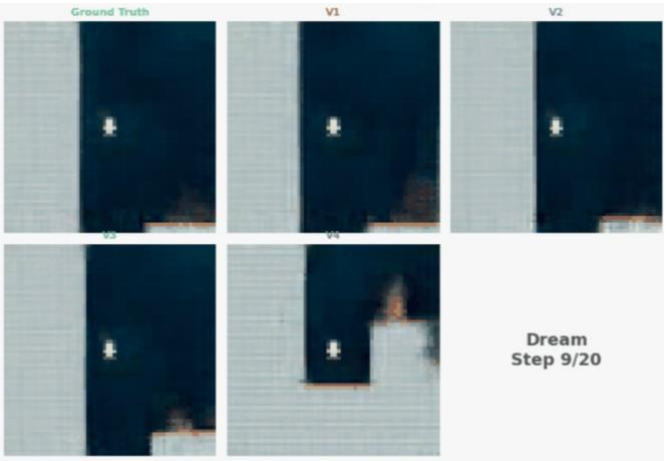

Fig. 8: CoinRun test set predictions at dream step 9 of 20. V1 and V2 maintain the correct scene structure, while V4 introduces significant visual artifacts in the upper right region, illustrating the compounding error problem at longer horizons for intermediate configurations.

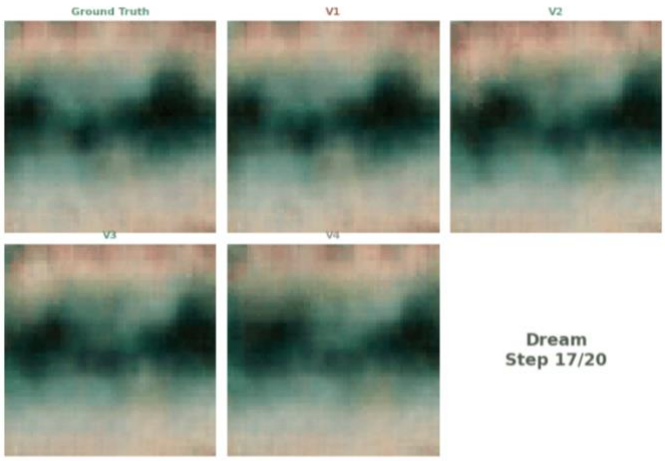

Fig. 9: StarPilot test set predictions at dream step 17 of 20. Ground truth (top left) compared with V1–V4 predictions. All models capture the broad spatial structure but lose fine detail, consistent with the high LPIPS values observed across StarPilot conditions.

V. Conclusion

Returning to the research question: can an IRIS-style world model [2] generalise to unseen level layouts in procedurally generated environments? The answer is yes, but the degree of generalisation depends on both the model configuration and the structural diversity of the environment.

The VQ-VAE tokenizer successfully compresses Procgen frames into discrete token grids with a reconstruction loss of 0.004 and over 90% codebook utilisation, confirming that discrete tokenization [3] is viable under procedural variability. All four transformer configurations produce coherent one-step predictions, validating that discrete visual tokens can be modelled autoregressively in this domain.

The capacity–regularisation ablation reveals that the optimal configuration is environment-dependent. On CoinRun, where level layouts vary substantially, V1's high capacity without regularisation leads to a generalisation gap of over 4 dB, while V2's heavy regularisation closes this gap almost entirely and outperforms V1 on unseen levels. On StarPilot, it’s hard to assess since there was a loss of information due to the VQ-VAE. However, regularisation alone is not sufficient to achieve both high accuracy and strong generalisation. Reducing the gap while maintaining high absolute performance would require training on significantly more level diversity, giving the model enough data to learn general dynamics without relying on regularisation to suppress memorisation.

The hypothesis was partially correct. V2’s moderate capacity with heavy regularization does generalise best on CoinRun, while V1 shows the largest generalisation gap. V3 and V4 were expected to find a middle ground, but they deteriorated faster over longer horizons, performing worse than both V1 and V2. This indicates that the capacity regularization trade-off [14] [19] is not a smooth spectrum, and intermediate setups can carry weaknesses from both ends.

Regarding research goals, O1 was achieved—the VQ-VAE tokenizer produces stable discrete representations. O2 was met as well—the transformer delivers coherent action-conditioned predictions. O3 was satisfied, showing clear differences across configurations. O4 was reached, as evaluation on unseen levels shows the generalization gap varies by environment and setup.

Future work could focus on training with more levels to see if greater layout diversity lessens the need for regularization. Applying this method to more Procgen games would deepen our understanding of the link between environment structure and ideal model setup. Using diffusion-based prediction, as in DIAMOND [11], could help tackle the compounding error problem that arises at longer horizons by working in continuous space and could address the loss of information caused by VQ-VAE. Adding a reinforcement learning controller would test if the learned dynamics assisted in policy learning in procedurally generated environments.

Lessons learned from this project include the importance of developing models in stages. Each version of the transformer learned from the failures of the one before it. We also recognised the value of tracking validation performance from the beginning, which the first version did not do. Additionally, we found that codebook collapse [13] in VQ-VAEs is more reliably fixed through changes in the design, like reducing the embedding dimension, rather than through training tricks, such as resetting inactive entries. Use of a larger dataset across more levels will help with generalisation, as the val loss remained high with the current setup. Overall, we learned that generalization in world models [1] involves more than just adding regularization. It depends on how model capacity, regularization strength, data diversity, and the structural properties of the environment interact.

REFERENCES

[1] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.

[2] V. Micheli, E. Alonso, and F. Fleuret, "Transformers are sample-efficient world models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[3] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[4] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes et al., "Genie: Generative interactive environments," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2024.

[5] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging procedural generation to benchmark reinforcement learning," in *Proc. 37th Int. Conf. Machine Learning (ICML)*, vol. 119, 2020, pp. 2048–2056.

[6] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter, "Diffusion models are real-time game engines," arXiv preprint *arXiv:2408.14837, 2024*.

[7] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Processing, vol. 13, no. 4, pp. 600–612, 2004.

[8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 586–595.

[9] K. Craik, The Nature of Explanation. Cambridge, U.K.: Cambridge Univ. Press, 1943.

[10] T. Winograd, "Procedures as a representation for data in a computer program for understanding natural language," Ph.D. dissertation, Dept. Math., MIT, Cambridge, MA, 1971.

[11] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storber, T. Pearce, and F. Fleuret, "Diffusion for world modeling: Visual details matter in Atari," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[13] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with VQ-VAE-2," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. on Machine Learning (ICML)*, vol. 37, 2015, pp. 448–456.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2015.

[17] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv preprint *arXiv:1308.3432*, 2013.

[18] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv preprint *arXiv:1606.08415*, 2016.

[19] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2019.

[20] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019.

[21] L. Kaiser, M. Babaeizadeh, P. Milos, B. Osinski, R. H. Campbell, K. Czechowski et al., "Model-based reinforcement learning for Atari," arXiv preprint *arXiv:1903.00374*, 2020.

[22] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.

[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[24] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv preprint *arXiv:1607.06450*, 2016.